AUDIO BEE

Technical Assessment

Web Scraping Intern – Practical Test

Position	Web Scraping Intern
Duration	60–90 minutes
Submission	Submit via email (or as a GitHub repo / zip) the following files: 1. `scraper.py` – Your main script 2. `output.json` – Extracted data 3. `pyproject.toml` – Project dependencies 4. `prompts.txt` – List of prompts used in the project 5. `recording.mp4` – Screen recording with voiceover explaining how selectors were found, how Cursor was used, and the overall code flow

If you miss any of these, your submission will be automatically rejected.

Overview

This practical test evaluates your ability to write browser automation scripts for web scraping. You will create a Playwright script in Python that extracts structured data from a Nepali news website. This assessment mirrors real tasks you will encounter at Audio Bee, where we build enterprise-grade data extraction solutions.

Note: You will need to use Cursor as an AI assistant for coding this project. See the AI-Assisted Coding Guide in the Appendix file.

Task Description

Target Website: https://ekantipur.com

Create a Playwright automation script in Python that performs the following:

Task 1: Extract Entertainment News

Navigate to the Entertainment (मनोरञ्जन) section and extract the **top 5 news articles** with the following data points:

Field	Description
`title`	The headline/title of the news article
`image_url`	URL of the post/thumbnail image (full URL)
`category`	The category/section label (e.g., "मनोरञ्जन", "बलिउड")
`author`	Author name (if available; use `null` if not found)

Task 2: Extract Cartoon of the Day

Locate and extract the "Cartoon of the Day" ("व्यंग्यचित्र") section with:

Field	Description
`title`	The cartoon's title or caption
`image_url`	Direct URL to the cartoon image
`author`	The cartoonist's name

Expected Output Format

Save your extracted data as a JSON file named `output.json` with the following structure:

```
{
  "entertainment_news": [
    {
      "title": "Article headline here",
      "image_url": "https://...",
      "category": "मनोरञ्जन",
      "author": "Author Name"
    },
    {
      "title": "Second article headline",
      "image_url": "https://...",
      "category": "बलिउड",
      "author": null
    }
  ],
  "cartoon_of_the_day": {
    "title": "Cartoon title",
    "image_url": "https://...",
    "author": "Cartoonist Name"
  }
}
```

Technical Requirements

- Use **Python 3.11+** with **Playwright**
- Use **uv** as the package manager
- Handle dynamic content loading (wait for elements properly)
- Implement error handling for missing elements
- Use appropriate selectors (prefer data attributes, then semantic HTML, avoid fragile XPaths)
- Add meaningful comments explaining your approach
- Output must be valid JSON saved to `output.json`

Setup Instructions

Step 1: Install uv (Package Manager)

macOS/Linux:

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

Windows (PowerShell):

```
powershell -ExecutionPolicy ByPass -c "irm https://astral.sh/uv/install.ps1 | iex"
```

After installation, restart your terminal or run:

```
source $HOME/.local/bin/env  # Linux/macOS
```

Step 2: Create Project

```
# Create project directory
mkdir ekantipur-scraper
cd ekantipur-scraper

# Initialize Python project with uv
uv init

# Add dependencies
uv add playwright

# Install Playwright browsers
uv run playwright install chromium
```

Step 3: Verify Installation

```
# Check Python version
uv run python --version

# Verify Playwright is installed
uv run python -c "from playwright.sync_api import sync_playwright; print('Playwright
```

Project Structure

Your project should look like this:

```
ekantipur-scraper/
├── pyproject.toml    # Project config (auto-generated by uv)
├── uv.lock           # Lock file (auto-generated)
├── scraper.py        # Your main script
└── output.json       # Generated output
```

Running Your Script

```
uv run python scraper.py
```

Helpful Playwright Methods (Python)

Here are some Playwright methods you will likely need:

```
# Navigation
page.goto("https://example.com")
page.click("a.nav-link")

# Waiting for elements
page.wait_for_selector(".article-card")
page.wait_for_load_state("networkidle")

# Selecting elements
elements = page.query_selector_all(".news-item")  # Multiple elements
element  = page.query_selector(".single-item")    # Single element

# Extracting data from single element
text = element.text_content()
href = element.get_attribute("href")
src  = element.query_selector("img").get_attribute("src")

# Looping through elements
cards = page.query_selector_all(".news-card")
for card in cards:
    title = card.query_selector("h2").text_content()
    image = card.query_selector("img").get_attribute("src")

# Handling missing elements safely
```

```
author_el   = card.query_selector(".author")
author_name = author_el.text_content() if author_el else None

# Using locators (recommended approach)
page.locator(".news-card").first.click()
titles = page.locator(".news-card h2").all_text_contents()
```

Tips for Success

1. **Inspect the website first** – Use browser DevTools (F12) to understand the HTML structure before writing code
2. **Start simple** – Get one piece of data working before extracting everything
3. **Use `headless=False`** – Watch the browser to debug issues visually
4. **Handle Nepali text** – The website uses Nepali (Devanagari script); use `ensure_ascii=False` when writing JSON
5. **Check for lazy loading** – Some images may load dynamically; you might need to scroll or wait
6. **Test incrementally** – Use `print()` statements to verify each extraction step
7. **Use try/except blocks** – Wrap element access in error handling for robustness

---

Submission Checklist

Before submitting, verify:

- ☐ Script runs without errors using `uv run python scraper.py`
- ☐ `output.json` contains valid JSON with proper encoding
- ☐ All 5 entertainment articles are extracted with complete data
- ☐ Cartoon of the Day data is complete
- ☐ Code has meaningful comments
- ☐ No hardcoded data (all data comes from scraping)

Submit the following files. This can be a GitHub repo or a zip file:

1. `scraper.py` – Your main script
2. `output.json` – Extracted data
3. `pyproject.toml` – Project dependencies
4. `prompts.txt` – List of prompts used in the project
5. `recording.mp4` – Screen recording with voiceover explaining how selectors were found, how Cursor was used, and the overall code flow